

Workflows + Agents + Apps

Implementation specification and phase-wise AI prompts — worklenz_local

Version 2 · 2026-07-04 · Source of truth: docs/APPS_AGENTS_PLAN.md

How to use this document

- Each phase (A–D) has a self-contained **prompt box**. Copy it into an AI coding agent (Claude Code) session opened at the repo root, one phase at a time, in order.
- Every prompt tells the agent to first read docs/APPS_AGENTS_PLAN.md and the reference files; the boxes carry only what is phase-specific.
- Finish a phase fully (acceptance criteria + quality gates pass) before starting the next.
- Section 2 (context) and section 3 (global guardrails) apply to every phase — the master preamble embeds them.

1. What is being built

Three layers on top of the existing PM+HR app:

APPS (App Center)	= external connectors: Webhook, Slack, Email(Resend), WhatsApp(Meta Cloud API), later MCP
WORKFLOWS	= Pabbly-style orchestration: Trigger -> Step -> Step -> ... (linear v1)
AGENTS	= reusable personas (skills bundle + data scope) invoked as a workflow step

- **Token economics (hard requirement):** AI (OpenRouter) is used ONLY at build time to compile a natural-language prompt into a workflow draft. Activated workflows run with ZERO AI tokens — deterministic skills (Postgres RPCs), template rendering, connector delivery. An explicit 'ai' step type (later phase) is the only runtime token consumer and must be visibly labeled.
- **Customizability (hard requirement):** everything selectable in the builder is a *capability descriptor* in one registry (key, kind, input_schema, output_schema, requires, source, runtime). Palette UI, config forms, field-mapping picker, executor dispatch and AI-compile enums all render from this registry. New step type = 1 descriptor + 1 executor handler. This is also the future MCP seam: a connected MCP server's tools become descriptors automatically.

Step types (v1)

Step	Config (jsonb)	Output into context
trigger.schedule	cron + tz → next_run_at	{fired_at}
trigger.event (Phase D)	internal events (task created / status changed / assignee added; HR: leave applied/approved)	event row data
agent	agent_id + param overrides	skills output object
condition	left / op / right over {{steps.*}} values	pass → continue, fail → stop (v1)
action	create_task, assign_member, add_comment, notify_user (reuse automations executor)	created ids
app	connection_id + template + recipients ({{...}} interpolation)	delivery status
human	assignee, ask (approve/reject + note), due, timeout (skip/stop)	{decision, note, acted_by, acted_at}; run pauses

Step	Config (jsonb)	Output into context
ai (Phase F)	model + prompt template	text (token-metered, opt-in)

Context object: every step writes its output under steps.<key>; later steps reference {{steps.s1.output.x}}. The builder shows a field-mapping picker built from each capability's output_schema.

2. Project context (read before any phase)

- Stack: Next.js 16 App Router + React 19 + AntD v5 + Tailwind; Supabase (Postgres, RLS-first, no REST backend) — clients call supabase-js + SECURITY DEFINER RPCs; TanStack Query hooks per feature dir; pg_cron for background jobs.
- Read first: docs/APPS_AGENTS_PLAN.md (full plan), AGENTS.md (Next.js 16 caveats — read node_modules/next/dist/docs before writing Next code).

Reference patterns to imitate (exact files):

Pattern	File
Config-table + executor + audit runs + RLS shape	supabase/migrations/20261009000000_automations_engine.sql
pg_cron sweep job (guarded DO-block registration)	supabase/migrations/20260701000000_phase7_... (materialize_recurring_tasks)
Skills-ready RPCs (report_*)	supabase/migrations/20260601000000_phase6_time_reportin g.sql
HR analytics RPC (jsonb)	supabase/migrations/20261005000000_hr5_analytics.sql (hr_org_analytics)
Availability RPCs	supabase/migrations/20261008000000_capacity_availability.s ql
In-app notification fan-out	create_notification() in phase5 migration
AI route skeleton + error mapping	src/app/api/ai*/route.ts + src/lib/ai/anthropic.ts
Service-role secret handling	src/app/api/account/delete/route.ts
Settings CRUD page + hooks pair	src/app/(app)/settings/clients/page.tsx + src/features/settings/use-clients.ts
Secondary nav registry	src/app/(app)/_lib/section-nav.ts

3. Global guardrails (every phase)

- RLS discipline: read = team/org member, write = admin (is_project_team_admin / is_org_admin style helpers). Run/audit tables written only by SECURITY DEFINER executors. Secrets tables: NO authenticated policies — service_role only, accessed via route handlers that first verify the caller with the SSR client.
- AI proposes, server validates, DB CHECK constraints enforce — model output is never trusted; enums constrained in the DB exactly like the automations engine.
- Migrations idempotent (create or replace / if not exists / drop policy if exists), timestamped after the latest existing migration; regenerate src/types/database.ts after each migration (or hand-add in generated format if regen hangs, matching existing precedent).
- Quality gates per phase: npm run typecheck, npx eslint on touched files (0 errors), npm run build, migration applied + smoke-tested on the local docker DB (supabase_db_worklenz_local, psql via docker exec) including RLS positive/negative tests, then an adversarial self-review of the diff.
- Never expose: service-role key, connector credentials, OPENROUTER_API_KEY to the browser. All model calls and secret reads happen in route handlers.
- Hinglish for user chat; ALL code, identifiers, comments, docs in English.

4. Master preamble (prepend to every phase prompt)

PROMPT — MASTER PREAMBLE (copy everything in this box to your AI coding agent)

You are implementing part of the "Workflows + Agents + Apps" feature in the worklenz_local repo (Next.js 16 + React 19 + AntD v5 + Supabase RLS-first; no REST backend - supabase-js + SECURITY DEFINER RPCs; TanStack Query feature hooks; pg_cron for background jobs).

Before writing any code:

- 1) Read docs/APPS_AGENTS_PLAN.md fully - it is the source of truth for the model, data model, execution engine, UI plan, capability registry, and MCP roadmap.
- 2) Read AGENTS.md and the relevant guides in node_modules/next/dist/docs/ (this Next.js version has breaking changes).
- 3) Study these reference implementations and imitate their patterns exactly:
 - supabase/migrations/20261009000000_automations_engine.sql (config tables, executor idiom, audit runs, RLS shape, updated_at trigger)
 - the pg_cron sweep in the phase7 migration (materialize_recurring_tasks)
 - src/app/api/ai/*/route.ts + src/lib/ai/anthropic.ts (route skeleton, error mapping)
 - src/app/api/account/delete/route.ts (service-role secret handling)
 - src/app/(app)/settings/clients/page.tsx + src/features/settings/use-clients.ts (CRUD UI)

Hard rules:

- RLS: read = member, write = admin; runs/audit tables written only by SECURITY DEFINER executors; secrets tables have NO authenticated policies (service_role only, via route handlers).
- AI output is never trusted: server validates, DB CHECK constraints enforce enums.
- Activated workflows must run with ZERO AI tokens (deterministic RPCs + templates + connectors).
- Migrations idempotent; regenerate src/types/database.ts (or hand-add types in generated format).
- Quality gates before you call the phase done: npm run typecheck && npx eslint (touched files, 0 errors) && npm run build; apply the migration to the local docker DB (docker exec -i supabase_db_worklenz_local psql -U postgres -d postgres < migration.sql) and smoke-test: happy path + RLS negative tests (cross-tenant user sees nothing / cannot write).
- Do NOT push to the cloud DB; leave `npm run db:push` to me.
- Commit only when I ask.

5. Phase A — Apps hub (App Center)

Goal: org-scoped connector storage + App Center UI + webhook & Slack providers working end-to-end.

PROMPT — PHASE A (copy everything in this box to your AI coding agent)

[paste MASTER PREAMBLE above, then:]

Implement Phase A - the Apps hub (App Center).

DB (one migration, e.g. supabase/migrations/<next_ts>_apps_hub.sql):

- app_connections(id uuid pk, org_id fk organizations cascade, provider text CHECK IN ('webhook','slack','email','whatsapp'), name text<=200, enabled bool default true, config jsonb CHECK object (NON-secret: url host label, channel name, from-address, phone display), created_by fk users set null, created_at/updated_at + updated_at trigger).
- app_connection_secrets(connection_id pk fk app_connections cascade, credentials jsonb, updated_at). RLS: app_connections select = is_org_member(org_id), insert/update/delete = is_org_admin(org_id); app_connection_secrets: enable RLS, create NO policies for authenticated (service_role only); grants accordingly.

Server (route handlers, following api/account/delete pattern - verify caller with SSR client, then service-role client for secrets):

- POST /api/apps/[id]/secrets -> upsert credentials (org admin only)
- POST /api/apps/[id]/test -> provider-specific test: webhook = POST {test:true} to url (from secrets/config); slack = post a "Connection test" message to the incoming-webhook URL. Return {ok, detail}. Map errors to friendly messages.

Client:

- src/features/apps/use-apps.ts: useAppConnections(orgId-scoped via active team's org), useCreateConnection, useUpdateConnection (metadata only), useDeleteConnection, useTestConnection + useSaveSecrets (fetch wrappers to the routes).
- UI /settings/apps (+ nav entry in src/app/(app)/_lib/section-nav.ts, icon "extension", label "Apps"): provider card grid (Webhook, Slack now; Email, WhatsApp as "coming soon" disabled cards), connected list (name, provider, enabled toggle, health dot from last test, Test button, Edit, Delete w/ Popconfirm). Connect/Edit modal: name + provider config form + a write-only credentials field (password input, never prefilled - placeholder "unchanged").
- Never fetch app_connection_secrets from the browser.

Acceptance:

- Org admin connects a Slack incoming webhook; Test button delivers a message to Slack.
- Generic webhook connection delivers a signed test POST (HMAC-SHA256 header X-Signature using a stored signing secret, if provided).
- Non-admin member: sees connections read-only, cannot create/edit (RLS negative test in psql).
- Cross-org user sees nothing (RLS negative test).
- Quality gates pass.

6. Phase B — Workflow engine core + capability registry + Builder UI

Goal: the heart. Registry, agents, linear runner, 3-pane builder, Run-now, run history — all with zero-token runs and in-app delivery.

PROMPT — PHASE B (copy everything in this box to your AI coding agent)

[paste MASTER PREAMBLE above, then:]

Implement Phase B - workflow engine core, capability registry, and the Builder UI.

Reference: docs/APPS_AGENTS_PLAN.md sections 2-6 (step types, data model, execution engine, UI plan, capability registry).

- 1) Capability registry (code-level v1): src/lib/workflows/capabilities.ts exporting typed descriptors {key, kind:'skill'|'action'|'app_action'|'logic'|'human', title, description, icon, category, input_schema (JSON Schema), output_schema, requires, runtime}. v1 skills map to existing RPCs: team_overview->report_team_overview, project_report->report_projects, member_report->report_members, timesheet->report_time_logs, hr_analytics->hr_org_analytics, availability->get_team_member_availability, overdue_tasks->new thin RPC (port the overdue/due-soon queries from src/app/api/ai/standup/route.ts into a SECURITY DEFINER RPC). Actions reuse the automations executor semantics (create_task via create_task RPC, add_comment, notify_user via create_notification).
- 2) DB (one migration): agents, workflows, workflow_steps, workflow_runs, workflow_step_runs exactly as in plan section 3 (skip human_actions + delivery_outbox - Phase C). CHECK-enum step_type IN ('agent','condition','action','app','human','ai') but the executor only implements agent/condition/action now (others raise 'not yet supported' into step error). RLS: workflows/agents/steps read = is_team_member, write = is_team_admin; runs/step_runs select = member, written only by the executor.
Executor RPCs (SECURITY DEFINER, imitate execute_task_automation's error idiom):
 - start_workflow_run(p_workflow_id, p_trigger_snapshot jsonb) -> creates run, calls advance.
 - advance_workflow_run(p_run_id) -> loops steps from current_position: execute step, merge output into context under steps.<step_key>, write step_run row; condition fail -> status 'success' with detail 'stopped by condition'; any error -> step_run.error + run status 'error' (never raise to caller). Skill dispatch: CASE on capability runtime, calling the mapped RPCs with params from config (team/org authority = the workflow's stored team).
 - Template interpolation: plain {{steps.key.field}} substitution implemented in SQL (simple regexp) for action/app configs - keep it dumb and deterministic.
- 3) Client hooks: src/features/workflows/ (use-workflows.ts CRUD, use-workflow-runs.ts, use-agents.ts CRUD, use-run-now.ts calling start_workflow_run rpc).
- 4) UI:
 - Sidebar: top-level "Workflows" section (section-nav.ts) with /workflows (list) and /workflows/agents pages.
 - /workflows list page: name, trigger chip, enabled Switch, last run status, run_count, "New workflow".
 - /workflows/[id] Builder - 3-pane layout per plan section 5.1: left palette rendered FROM the registry (search + categories; each item icon/title/one-liner; drag or click-to-add); center vertical stepper canvas (dnd-kit reorder; step cards show icon, key, config summary, incomplete-config warning badge); right inspector with config form AUTO-GENERATED from the capability input_schema (build a small JSON-Schema->AntD form renderer supporting string, number, boolean, enum select, uuid-picker fields (team members, projects, agents via lookup hooks)) + "Insert data" picker listing upstream steps' output_schema fields -> inserts {{steps.key.field}} at cursor; bottom Test console drawer: "Test run" executes start_workflow_run and polls the run + step_runs, rendering per-step status/input/output JSON + "0 AI tokens used" note.
 - /workflows/agents: gallery cards + editor (skills picker from registry, param forms via the same renderer, "Preview output" button running the skills read-only via a preview RPC or the run path against a throwaway run).
 - Run history tab on /workflows/[id]: runs table -> expandable per-step timeline.

Acceptance:

- Build "Weekly HR pulse (manual)" in the Builder: agent step (HR Analyst: hr_analytics + availability) -> condition (steps.s1.leave_pending > 0) -> action (notify_user a chosen member with an interpolated message). Run now -> notification received with real numbers; run history shows all step inputs/outputs; zero AI involved.
- Condition-fail path stops cleanly (run success, detail says stopped).
- A step misconfigured -> run status error, other workflows unaffected.
- RLS negative tests: cross-team user sees nothing; member cannot create workflows (admin only).
- Quality gates pass.

7. Phase C — Scheduler, human-in-the-loop, external delivery

Goal: unattended runs + approvals inbox + delivery through Phase-A connectors via an outbox.

PROMPT — PHASE C (copy everything in this box to your AI coding agent)

[paste MASTER PREAMBLE above, then:]

Implement Phase C - scheduling, the human step, and external delivery.

Reference: docs/APPS_AGENTS_PLAN.md sections 3-5 (human_actions, delivery_outbox, sweep).

1) DB migration:

- human_actions + delivery_outbox tables per plan section 3.
- Scheduler: run_due_workflows() SECURITY DEFINER RETURNS integer - sweeps workflows where enabled AND trigger_type='schedule' AND next_run_at <= now(): start_workflow_run(), advance next_run_at from the cron expr (implement a minimal cron-next-occurrence SQL helper supporting m h dom mon dow with numbers, *, lists, ranges; timezone via config.tz). Register ONE guarded pg_cron job 'run-due-workflows' every 5 minutes (copy the phase7 DO-block idiom).
- human step in advance_workflow_run: create human_actions row + create_notification to the assignee -> set run status 'waiting_human', stop the loop. decide_human_action(p_action_id, p_decision, p_note) SECURITY DEFINER: assignee-only check, records decision, merges {decision,note,acted_by,acted_at} into context under the step key, resumes advance_workflow_run. Timeout handling inside the sweep: expired due_at -> per config 'skip' (merge {decision:'expired'}) or 'stop' (run status 'stopped').
- app step in advance_workflow_run: render payload (template interpolation) -> insert delivery_outbox(pending) -> continue (do not block the run on delivery).

2) Delivery poller: POST /api/workflows/deliver (protected by a shared secret header DELIVERY_POLL_KEY from env, so it can be invoked by an external cron/uptime pinger, and also opportunistically after Run-now): service-role reads pending outbox rows (limit 20, oldest first, attempts < 5), per provider: webhook = POST payload (+ HMAC if signing secret); slack = incoming-webhook text (render report as Slack-friendly markdown); email = Resend API (RESEND_API_KEY from connection secrets; simple HTML wrapper around the rendered report). Update status sent/failed + attempts + last_error.

3) UI:

- "My approvals" page + navbar badge (count of pending human_actions for me): context card (workflow name, step message with interpolated data, requested note), Approve / Reject + note -> decide_human_action rpc -> run resumes; show resulting run status inline.
- Builder: human + app step types now fully configurable (assignee picker, due/timeout; connection picker filtered by provider, recipients, template textarea with Insert-data).
- Workflow list: next_run_at column for scheduled workflows.

Acceptance:

- Weekly HR pulse with schedule 5 minutes ahead + human approval + slack delivery: fires unattended, pauses on approval (badge appears), approve -> resumes -> Slack message arrives with the HR manager's note interpolated; email + webhook providers verified the same way.
- Reject path stops the run with status 'stopped'.
- Timeout skip path verified (short due).
- Outbox failure (bad webhook URL) -> failed row with attempts/last_error; sweep keeps working.
- RLS: only the assignee can decide their human action (negative test).
- Quality gates pass.

8. Phase D — AI builder (OpenRouter), WhatsApp connector, event triggers

Goal: prompt-to-workflow compile, WhatsApp delivery, and event-triggered workflows.

PROMPT — PHASE D (copy everything in this box to your AI coding agent)

[paste MASTER PREAMBLE above, then:]

Implement Phase D - the AI workflow builder, the WhatsApp provider, and event triggers.

Reference: docs/APPS_AGENTS_PLAN.md sections 5-9 (AI assist, MCP-ready registry, research appendix for WhatsApp/OpenRouter facts).

- 1) OpenRouter client: src/lib/ai/openrouter.ts mirroring src/lib/ai/anthropic.ts's surface (getClient/mapError/extractJson) but via plain fetch to `https://openrouter.ai/api/v1/chat/completions` with Authorization: Bearer OPENROUTER_API_KEY, HTTP-Referer + X-OpenRouter-Title headers, response_format {type:'json_schema', json_schema:{strict:true, schema}}, provider:{require_parameters:true}, and a models fallback array (env OPENROUTER_MODELS, default 2-3 capable models). Friendly 503 when the key is missing (copy the anthropic.ts missing-key mapping).
- 2) Compile endpoint POST /api/workflows/compile {teamId, prompt, existingWorkflowId?}: SSR auth -> RLS context (registry capabilities, team members, agents, connected apps) -> strict json_schema constraining output to: trigger (schedule cron/tz or event type from the supported enum), steps[] with step_type + capability key + config matching each capability's input_schema (embed per-capability param enums where feasible), template strings using only {{steps.*}} refs that exist. Server-side validation pass after parse: unknown capability -> drop + warning; unknown field refs -> warning list. Response = draft {workflow, steps, warnings} - DO NOT insert; the client shows it in the Builder as a diff/preview (new steps highlighted) for the user to accept (then normal create/update mutations run) or tweak. UI: "AI assist" button in Builder top bar -> prompt modal -> preview.
- 3) WhatsApp provider (Meta Cloud API, per research appendix):
 - Connection config: phone_number_id, waba_id, template_name, template_lang; secrets: permanent access token. Test = GET the phone number info via graph API.
 - Outbox delivery: POST graph.facebook.com/v23.0/{phone_number_id}/messages with type template, body params from the rendered report summary (short line values - template params reject newlines; keep a params[] array mapping) + optional URL button param.
 - Recipients: per-user WhatsApp numbers - add users.whatsapp_number (or a user_channels table) captured in Settings > Profile with an explicit consent checkbox; recipients picker only lists members with consented numbers.
 - Statuses webhook (optional if time permits): GET verify handshake + POST handler storing delivered/read/failed onto the outbox row (X-Hub-Signature-256 verification).
- 4) Event triggers: extend the automations-engine matcher functions (or add parallel triggers) so task events (created / status_changed / priority_changed / task_completed / assignee_added) can also enqueue workflow runs: a workflow_event_index on workflows (trigger_type='event', enabled) matched by project's team; call start_workflow_run with the event row as trigger_snapshot. Reuse the existing app.automations_depth guard so workflow actions cannot re-trigger event workflows recursively.

Acceptance:

- Prompt "Har Monday 9 baje HR report banao, HR manager approve kare, phir founder ko WhatsApp jaye, 5+ overdue pe PM ko task do" compiles to a valid 5-step draft; accept -> identical to the hand-built Phase B/C workflow; compile cost = exactly one OpenRouter call.
- WhatsApp test connection + a real template delivery to a consented number works (document the Meta setup steps in docs/WHATSAPP_SETUP.md as part of this phase).
- Event workflow: "when task completed -> notify member via Slack" fires from a real status change, and cannot loop (guard verified).
- Compile with a nonsense prompt returns a clean 422 with guidance, never a broken draft insert.
- Quality gates pass.

9. Phase E/F (outline — prompt them when you get there)

- E: MCP client — 'Connect MCP server' provider (OAuth 2.1 / token), tools/list discovery → capability descriptors (source mcp:<connection_id>), mcp_tool step executor via outbox; Router/Delay step types;

Templates gallery.

- F: metered 'ai' step type; inbound triggers (WhatsApp replies / incoming webhooks → workflow); chat-with-agent; our own MCP server exposing skills (.well-known server card).

10. Decisions still open (answer before Phase B ideally)

- Agent/workflow scope: team-level only, or org-level allowed for HR agents? (plan §10 Q1)
- OpenRouter key: app-level single key (recommended) vs workspace BYOK. (Q2)
- WhatsApp recipients: members-only vs external numbers with separate consent. (Q3)
- Human step v1: approve/reject+note only, or form inputs too. (Q5)
- Existing automations engine: keep separate as quick rules (recommended) vs merge. (Q6)